

COSC 3430 • ARTIFICIAL INTELLIGENCE

End-to-End Machine Learning Project

Part 1: look at the big picture

ROADMAP

Ten steps, over several code-along lectures

- 1 Look at the big picture
- 2 Get the data
- 3 Explore and visualize the data
- 4 Create the test data
- 5 Prepare the data for learning
- 6 Select a model and train it
- 7 Fine-tune the model
- 8 Present your solution
- 9 Launch the production system
- 10 Monitor and maintain it

Today: step 1, the big picture.

SOURCE

Géron, Chapter 2: about 60 dense pages

The book

Hands-On Machine Learning with Scikit-Learn and PyTorch. Very dense, with many side alleys and tangents. We follow the main road.

The notebook

Géron's notebook (mostly code) runs in Colab. Go through it on your own:

tinyurl.com/geron-2-ipynb

QUESTION

Why start with the data, and not with the model?

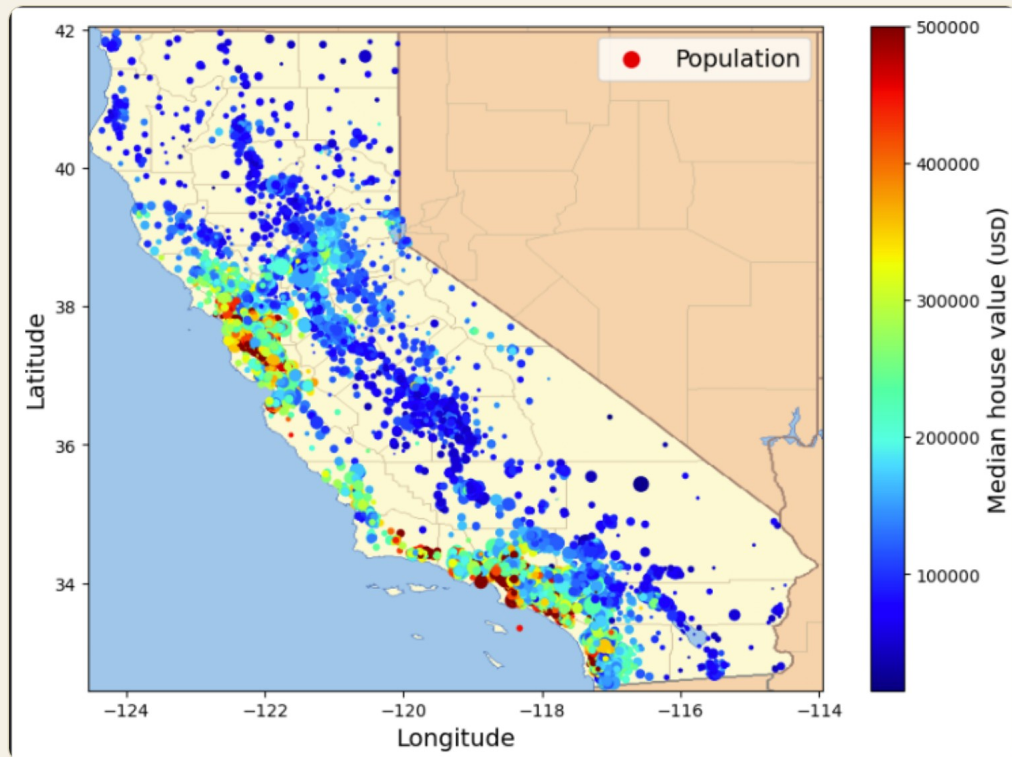
ANSWER

Machine learning learns rules *from data* and bakes them into a model that predicts on *new data*.

So the data takes the center role: not the model, not the language, not the infrastructure.

TARGET DATA

California Housing Prices, 1990



Origin: US Census Bureau, 1990 (also on Kaggle)

Metrics: population, median income, median house value and more, per district (block group)

Why: realistic (though outdated) and verifiable

STEP 1 • CHECKLIST

Frame the problem, look at the big picture

- 1 Define the objective in business terms
- 2 How will your solution be used?
- 3 Current solutions or workarounds?
- 4 How to frame it (supervised/unsupervised, online/offline ...)?
- 5 How should performance be measured?
- 6 Is the measure aligned with the business objective?
- 7 Minimum performance needed?
- 8 Comparable problems? Reusable experience or tools?
- 9 Is human expertise available?
- 10 How would you solve it manually?
- 11 List the assumptions made so far
- 12 Verify assumptions if possible

QUESTION

Is there a business need for machine learning?

Yes: a model can replace slow, costly expert estimates

Model output

A prediction of a district's median house price

Goal

Decide whether an area is worth investing in

Data pipeline

Output feeds another ML system. Upstream = near the data; downstream = near the client.

Current solution

Experts apply complex rules: costly, slow, error-prone

QUESTION

How would you frame this as an ML problem?

Supervised, unsupervised or reinforcement? Classification or regression? Batch or online?

ANSWER

Supervised, regression, batch

Supervised

Every row (instance) comes with the expected output: the district's median house price, the **label**.

Regression

Predict one continuous number from many features: **multiple** (predictors), **univariate** (predicted) regression.

Batch

Static, small data (1.42 MB) that doesn't change fast: train offline on all of it, then launch.

QUESTION

What do you need to assess learning success?

A performance measure: Root Mean Squared Error

$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}^{(i)}) - y^{(i)})^2}$$

A **loss** or **cost function**: one number that says how far the predictions are from the truth.

Symbol	Meaning
m	number of instances (rows), $i = 1 \dots m$
$\mathbf{x}^{(i)}$	feature vector of instance i : all feature values except the label
$h(\mathbf{x}^{(i)})$	the model's prediction for instance i : a single number
$y^{(i)}$	the actual label of instance i
$\mathbf{X}$	feature matrix: one row per instance, row $i = \mathbf{x}^{(i)}$ transposed

RMSE for five districts

1. Put five districts in a **DataFrame**: features plus the label
2. Write down **actual** and **predicted** prices as NumPy arrays
3. Compute RMSE by hand: square, mean, square root
4. Check it with scikit-learn's **root_mean_squared_error** (v1.4+)

QUESTION

Why use RMSE? And what do all performance measures have in common?

ANSWER

Interpretable units, one number

Why RMSE

It's in the same units as the target. Our five districts: predictions are off by about **\$11,464**, a number anyone can read.

What they share

Every measure is a function of **$\mathbf{X}$** and **$\mathbf{h}$** : it aggregates the gaps between predictions $h(x(i))$ and labels $y(i)$ into one number.

RMSE squares the errors, so large errors weigh heavily. **MAE** (mean absolute error) weighs every error equally, so it's more forgiving of outliers.

The problem is framed

- Predict a district's median house price to guide investment
- Supervised, batch, multiple univariate regression
- Scored with RMSE, in dollars

Next: get the data, and take a first look at it.